FUDGEL

Language Architecture Specification

v3.1 — Draft

ProtoSpeech Foundation
fudgel.org

0. What Fudgel Is

Fudgel is a meta-language of intent. It is not a general-purpose programming language. It does not compute, render, allocate memory, or manage time. It describes the structure, logic, and data flow of an application in three distinct sub-languages — each with a single job — and compiles that description directly to bare-metal instruction sets with no intermediate abstraction layer.

The architecture is three layers, each completely agnostic to the others:

DSL	Layer	Single Responsibility
`Bija`	State	Stores all state as a static entity tree. No logic, no operations. Human-readable names are discarded at runtime — every entity becomes a fixed memory address.
`Rumus`	Logic	Pure, finite functions that operate exclusively in registers. Reads no memory, writes no memory. Everything it touches is gone when it returns.
`Viesti`	Wiring	Connects Bija addresses to Rumus functions and writes results back to Bija. Supports sequential and parallel execution through a two-symbol syntax.

Everything else — type checking, validation, timing, rendering, layout, animation, file I/O, encryption — is handled by domain-specific libraries. Each library is a domain-scoped semantic abstraction that compiles directly to Bija entities, Rumus functions, and Viesti signals. There is no distinction at the machine level between library code and author code.

Fudgel programs are deterministic by construction. Because Bija is fully resolved at load time, Rumus functions are pure, and Viesti resolves signals in declaration order, the same input state always produces the same output. Non-determinism can only be introduced explicitly through a library.

> *Core Law: if you need logic, write a Rumus function. If you need to remember something, write it to a Bija entity. Viesti connects the two. Nothing else exists.*

0.1 Design Philosophy

Fudgel exists because the dominant software stack was never designed to be a universal application runtime. It became one through historical accident. The result is that applications with completely different constraints end up built on infrastructure designed for document delivery over a network.

The standard response has been more abstraction: frameworks, transpilers, runtimes-within-runtimes. These abstractions treat real symptoms — GC prevents use-after-free, ORMs prevent injection — but they are applied universally at a cost paid even when the original problem does not apply. A local game does not need GC. A simulation does not need an ORM. The tax is paid regardless.

Fudgel is closer to a Turing-complete algorithm description format than a conventional programming language. It says not "here is how to program this CPU" but "here is the algorithm — these are the steps, these are the dependencies, these can run in parallel, these cannot." Whether any given piece of hardware can execute it is a separate question answered by whoever builds the runtime for that substrate. The same source could run on RISC-V today, a GPU tomorrow, and a substrate that does not exist yet next decade — as long as someone has written the library that maps Rumus operations to that substrate's primitives. The algorithm is the stable thing. The execution target is not.

This is why the library system is the extension point rather than the language. Ehto and Primi are today's answers to what correctness means for current hardware. When computation changes, the language does not need to change. New memory models get new libraries. New instruction sets get new emitter libraries. The description of the computation — the Bija state, the Rumus transformations, the Viesti dependency graph — stays exactly as written. Fudgel is substrate-independent by design, not by accident.

0.2 The False Tradeoff

Language design has historically operated on an assumed spectrum: human-readable on one end, machine-efficient on the other. Fudgel's claim is that this tradeoff is false. A sufficiently honest representation of intent is also a sufficient representation for direct compilation, because intent at the right level of granularity is exactly what the machine needs.

Bija describes memory layout. Rumus describes operations. Viesti describes execution order and data movement. That is not a simplification of what a computer does — it is an honest description of it in legible syntax. Legibility does not require abstraction. It requires good notation.

The key reframe: the decisions Fudgel asks a developer to make are not performance decisions. They are truth claims about the program. `~` means *this depends on that*. `~~` means *these have no dependency on each other*. `{immutable}` means *this value never changes*. `!5` means *these instances are identical in structure*. The hardware uses these truths to execute correctly and efficiently — but the developer is not managing the hardware. They are describing their program accurately.

0.3 Error Handling and Safety

Fudgel is designed to hard crash on invalid runtime state. There is no exception system, no error monad, no try/catch. If a program reaches an invalid state at runtime, it stops. This is deliberate: runtime error handling is a secondary system bolted onto a primary system that already failed to prevent the error. Preventing the error is the correct investment.

Prevention happens at two levels. First, the library compilation pass — Ehto and other libraries inspect the Bija entity tree before the PAF is emitted, validating constraints and flagging missing references as compile-time errors. Second, the IDE's visualiser surfaces structural issues before a build is attempted. The hard crash is a last resort, not a first line of defence.

0.4 The IDE is Load-Bearing

The Fudgel IDE is not a convenience tool. It is a core part of the design thesis. The signal graph, the entity tree, and the compilation pass results are all structures that are intuitive when visible and intimidating when read as text. The WYSIWYG visualiser is what makes hardware-level explicitness accessible without being assembly-level painful.

The language and IDE should ship together. A language that requires a visualiser to be ergonomic but ships without one will be correctly perceived as hostile.

0.5 Target Domain

Fudgel's natural first audience is game and interactive application development. That community already thinks in ECS, already cares about determinism and performance, and is already frustrated with the abstraction overhead of existing engines. The longer-term target is any application that does not require the web stack's specific capabilities but ends up using it anyway due to ecosystem gravity.

0.6 Fudgel and AI Tooling

Fudgel's signal graph is the actual runtime topology, not an abstraction over it. Every data dependency is a declared edge. Every state mutation is a named Viesti signal. There is no hidden control flow to infer. A model reading a Fudgel application sees the real execution structure, not a representation of it.

What AI coding tools actually do is manage dataflow. A model pattern-matches on relationships between inputs, transformations, and outputs. Fudgel makes that graph literal. A Viesti graph is structurally closer to how a transformer reasons about information movement than a for-loop is. This makes Fudgel a better substrate for AI reasoning than almost anything currently in production use.

1. The Trinity

Fudgel is composed of exactly three sub-languages, each with a single responsibility. No sub-language can perform the role of another. This separation is enforced by the syntax of each sub-language: a Bija file contains no expressions, a Rumus function contains no storage writes, a Viesti graph contains no conditionals.

DSL	Single Responsibility	What it cannot do
`Bija`	Store state and structure	Execute logic or route data
`Rumus`	Compute pure functions	Store state or route data
`Viesti`	Route data between addresses	Store state or execute logic

A file's type is defined by its Trinity imports:

File Type	Required Imports	Description
Application	@Bija @Rumus @Viesti	Complete executable. First signal under @Viesti is the entry point.
Library	@Bija @Rumus	Templates and functions. No @Viesti — wiring is the consumer's concern.
Data file	@Bija	Pure state declaration. No logic, no signals.

2. Bija — The State Layer

2.1 Purpose

Bija is the entire heap of a Fudgel application. It is the only place where state persists across function calls. All Rumus functions read from and write to Bija entities. All Viesti signals reference Bija addresses.

At runtime, every human-readable name in a Bija file is discarded. The load-time pass assigns each entity and property a fixed memory offset. All subsequent references become raw pointer arithmetic.

Bija is designed to describe not just what exists but what should be derived from it — without functions. Templates define a shape once. Instantiation copies it. The multiplier stamps out N copies. Positional shorthand overrides values inline. All of this resolves at parse time. The runtime sees only the fully expanded result: a flat set of named entities at fixed addresses.

This is the key distinction between Bija and general data formats like JSON or PKL. Those formats describe what exists. Bija describes what exists and how to derive everything else from it, purely through structure, with no runtime cost.

2.2 Core Rules

- Everything is entities and properties.
- Every property has exactly one value.
- Indentation denotes hierarchy.
- The parser has no type system, no logic, and no operations. Meaning arises from structure, not syntax.
- **Bija values are always atomic.** A value is a literal, a reference, a shorthand token, or `empty`. It is never an expression. `damage: #attack + 20` is a syntax error.
- All entities are mutable by default. The {immutable} flag is a hint to the emitter, not a language guarantee. Immutable entities are inlined as immediates at compile time and assigned no memory address.
- All shorthand — commas, !, positional pipes, inline sub-entity — resolves at parse time. Nothing from shorthand syntax survives into the PAF.

Every piece of shorthand syntax in Bija follows the same methodology: the parser recognises a token pattern and mechanically expands it into the equivalent verbose form. There is no evaluation, no arithmetic, no logic — only pattern recognition and expansion. If a shorthand token does not match its expected pattern, it is a syntax error. The parser does not attempt to infer intent.

2.3 Syntax Reference

Entities

```
EntityName;
  SubEntity;
```

Named containers. End with a semicolon. Indentation denotes parent-child hierarchy.

Properties

```
propertyName: value
```

Always name: value. Belongs to the most recently declared entity. Value is a literal, a reference, a shorthand token, or empty. Never an expression.

Empty Value

```
propertyName: empty
```

The explicit empty value. No inference occurs. A property with no value is not null — it is empty by declaration.

References —

```
#EntityName
#EntityName.property
#list.0        (ordered index)
```

`#` means these two things are the same thing. A semantic alias for the author's benefit. The compiler resolves it to the same memory address as the target and removes it. No indirection survives into the compiled output. Digits are reserved — property names must begin with a letter.

Value Copy — =

```
displayHealth: =Player.health
```

Copies the referenced property's value to this property at parse time. The `#` prefix is not used — `=` already implies a Bija property reference. The source is forgotten after copying. Not an alias, not a live link. If the source changes at runtime, this property does not change with it.

Instantiation

```
NewThing: #TemplateName
```

Copies the referenced entity's full structure at parse time, including all sub-entities and their properties. Not a link. The copy is fully independent after instantiation.

Positional Shorthand — | v1, v2 |

```
(inline sub-entity — declare structure and values in one line)
Stats | strength, agility, luck |;

(template override — replace template values for this instance only)
Sword: #TemplateItem | 15, 3 |
```

The `| value1, value2 |` pipe shorthand assigns property values positionally. The **semicolon** distinguishes the two uses: `Entity | p1, p2 |;` is an entity declaration; `prop: #Template | v1, v2 |` is a property with a reference override. On a template instantiation, positional values override the corresponding template properties for that instance only.

```
TemplateItem;
  value: 0
  weight: 1
  durability: 100

(override value and weight — durability stays 100)
Sword: #TemplateItem | 15, 3 |

(override all three)
Shield: #TemplateItem | 5, 8, 200 |
```

Multiplier — !N

```
enemies: #Enemy !5

(compiler expands to)
enemies;
  Enemy_0: #Enemy
  Enemy_1: #Enemy
  Enemy_2: #Enemy
  Enemy_3: #Enemy
  Enemy_4: #Enemy
```

Stamps out N independent instances at parse time. The count must be a literal integer — it is a repetition token, not a number being evaluated. Each instance is fully independent in compiled memory, addressable as `#enemies.0` through `#enemies.4`. Combinable with positional shorthand for homogenous sets:

```
wave: #Enemy !5 | 200, aggressive |
```

If each instance requires different values, write them explicitly.

Anonymous Lists — - and *

```
(ordered — author writes)
loot: -
  gold
  gem
  potion

(parser expands to)
loot;
  0: gold
  1: gem
  2: potion

(unordered — author writes)
flags: *
  active
  visible
  collidable

(parser expands to)
flags;
  active: empty
  visible: empty
  collidable: empty
```

The `-` marker creates numbered sub-entities with their values. The `*` marker creates named sub-entities with empty values — the names are the data, not labels for values. Both resolve at parse time. Neither construct exists in the compiled output. Ordered items are addressable by index: `#loot.0`, `#loot.1`, etc.

Metadata Booleans — {flag1, flag2}

```
health: 100 {required, unique}

(expands to)
health: 100
  required: true
  unique: true
```

Only booleans belong inside {}. Presence equals true. Bija has no privileged boolean keywords. The words required, unique, immutable, and any others are not part of the Bija language — they are ordinary boolean sub-entities. Their meaning is supplied entirely by the library that reads them during the compilation pass.

Literal Strings — [...]

```
propertyName: [literal text content
that may span multiple lines]
```

Square brackets denote a literal value. No parsing occurs inside. May span multiple lines.

Comments

```
(this is a comment and is ignored by the parser)
```

Parentheses delimit comments in all three DSLs. Comments may appear anywhere, including inline. Nesting is supported.

2.4 Reference Semantics

Syntax	Name	Behaviour
#Entity.property	Alias	Compiler resolves to same address. Reference removed. No runtime indirection.
=Entity.property	Value copy	Value copied at parse time. Source forgotten. Independent at runtime.

Neither syntax creates a live link. Bija is fully static after load. The only mechanism that reads or writes Bija state at runtime is a Viesti signal.

2.5 @ Declarations and File Types

`@` has one universal meaning: bring this into scope. It applies to the Trinity DSLs, compilation-pass libraries, and domain libraries. Every Fudgel file must declare its imports explicitly. Trinity imports also act as section markers — declarations following a Trinity import belong to that layer's scope. Import order is declaration order is compilation-pass order.

```
@Bija
@Ehto

Player;
  health: 100 {required}

@Rumus
? clamp{x, min, max};
  belowMin: x < min ? min : x
  result: belowMin > max ? max : belowMin
  ~result

@Viesti
Player.health ~ clamp ~ Player.health
```

2.6 Full Bija Example

```
@Bija
@Ehto

Player;
  name: Kael
  health: 180 {required}
  mana: 90 {required}
  startingHealth: =Config.maxHealth
  inventory;
    loot: -
      gold
      gem
      potion
    equippedItem: #loot.0
  stats;
    strength: 10
    agility: 8
```

```
Enemy;
  health: 80
  behaviour: passive

TemplateItem;
  value: 0
  weight: 1

Sword: #TemplateItem | 15, 3 |
  durability: 100

Scene;
  basicEnemies: #Enemy !5
  eliteWave: #Enemy !3 | 200, aggressive |

Config;
  maxHealth: 100 {immutable}
  version: 1 {immutable}

UI;
  shownHealth: =Player.health
```

3. Rumus — The Logic Layer

3.1 Purpose

Rumus is a pure transformation notation. A Rumus function takes named inputs, applies a transformation, and exposes outputs. It has no side effects, no memory allocation, no loops, and no branching. It describes what a computation is, not what a substrate does to perform it.

Rumus core defines the shape of transformation — bind, compose, apply, select — without assuming anything about what kind of values are being transformed or what hardware will execute the transformation. The numeric operators, the arithmetic primitives, the instruction mappings are substrate assumptions that belong in a target library, not the core language.

The current working target is @rumus-riscv, which declares numeric arithmetic and comparison as its primitive operation set and maps them to RISC-V instructions. A file targeting a different substrate imports a different target library instead. The Rumus functions themselves do not change. What changes is what the emitter does with them.

There is no optimisation pass. Performance is a direct product of design quality. The compiler's job is honest translation, not cleverness. Structural performance gains fall out of the architecture — flat memory, pure functions, explicit signal graph.

3.2 Rumus Core — Transformation Primitives

Rumus core defines the minimum set of operations from which all computation can be expressed, independent of substrate. These are mathematical primitives, not hardware instructions.

Primitive	Description
bind	Give a value a name for use within a function scope. Parameters are bindings.
apply	Pass a value to a function and receive its output. Function calls are applications.
compose	The output of one function becomes the input of another. The ~ output feeds the next binding.
select	Given multiple values, produce one. The conditional shorthand expands to this.

These four primitives are sufficient for Turing completeness without recursion, given that iteration is handled by Viesti signal cycles. Everything above this level is either a target library primitive or ergonomic shorthand that expands to compositions of these four.

3.3 @rumus-riscv — Current Working Target

The @rumus-riscv target library declares numeric computation as a first-class primitive set for RISC-V execution. Importing it makes the following operations available as atomic Rumus primitives.

Rumus operation	RISC-V instruction
x + y	ADD
x - y	SUB
x * y	MUL
x / y	DIV
x % y	REM
x = y	BEQ
x < y	BLT
a ? b : c	Branch + label

The bitwise operations (&, |, ^, ~, >>, <<) are hardware-specific and belong in a systems target library. They are not part of the standard operation set. The ternary a ? b : c is shorthand for a Rumus core select operation — it expands at emit time. It is not a branching primitive. Rumus has no branching. Branching is Viesti's job.

3.4 Standard Operation Set

The Rumus standard operation set is defined by what a programmer naturally reaches for when expressing a transformation. It covers five domains, available in any Rumus file without explicit import.

Arithmetic

Operator	Description
x + y	Addition
x - y	Subtraction
x * y	Multiplication
x / y	Division
x % y	Remainder — divisibility checks and wrapping
-x	Unary negation — distinct from binary subtraction

Comparison

Operator	Description	Notes
x = y	Equal	Single = is equality in Rumus, not assignment
x != y	Not equal	
x < y	Less than	
x > y	Greater than	
x <= y	Less than or equal	
x >= y	Greater than or equal	

> *Equality in Rumus is a single =. It is a comparison, not assignment. Bija uses = for value copy; Rumus uses it for equality testing inside expressions.*

Boolean

Operator	Description
a & b	Boolean AND — both true
a | b	Boolean OR — at least one true
!a	Boolean NOT — inverted truth value

Conditional

```
a ? b : c
```

Produces a value — does not route behaviour. Valid only when both branches produce values of the same kind through pure transformation of inputs. Any selection that routes to meaningfully different behaviours belongs in Viesti.

Formal Logic — Assertions

Assertions are statements about relationships between truth values, scoped to function evaluation. They are not expressions — they are constraints that must hold. If an assertion fails at function evaluation, execution halts with the function path and the specific assertion that failed.

Operator	Name	Meaning
a -> b	Implication	If a is true, b must be true
a <-> b	Biconditional	a is true exactly when b is true
a ^ b	Exclusive or	One or the other, never both
a !& b	NAND	Not both simultaneously true — mutual exclusion
a !| b	NOR	Neither is true — joint absence
a <- b	Converse implication	If b is true, a must be true

Assertions make IDE function cards into interactive proof tools. When a developer runs a function in the IDE test tab, all assertions fire against those inputs. An assertion that fails immediately surfaces the exact implication that broke, and with what values. The assertions are a living spec for the function, co-located with the implementation, executing every time it runs — not tests in a separate file, not comments that go stale, but executable domain knowledge.

3.5 Function Declaration and Body

```
? functionName{param1, param2};
  intermediate: param1 + param2
  param1 > 0 -> intermediate > param1   (formal logic assertion)
  anotherStep: intermediate * 2
  ~anotherStep
```

The ? prefix marks a Rumus function entity. Parameters use {} boolean shorthand — each expands to a sub-entity with value true. Viesti overwrites them with actual values before execution. A function cannot run unless all parameters have been supplied.

Body properties are named intermediates (let-bindings) evaluated top to bottom. Body property names are discarded at emit time and become positional register slots. Body property count directly determines register demand. A function body that exceeds available register capacity is undefined behaviour — decompose the function.

3.6 Output Declaration — ~

```
~value

~value1, value2
```

~ declares which values are exposed to Viesti. These are the only externally addressable values on a function entity. A Viesti signal may only target a parameter address or a ~ declared output address. All other properties are ephemeral and do not exist in the PAF.

3.7 The Function Address Surface

Address	Description
functionName.x (parameter)	Viesti writes here to supply data. Overrides the default true before execution.
functionName.lo (~ output)	Viesti reads here to route the result onward after the function returns.

3.8 Calling Another Function

```
? normalise{x};
  clamped: #clamp | x, 0, 100 |
  result: clamped / 100
  ~result
```

A Rumus function may call another using Bija instantiation syntax. The called function's ~ output values become named values in the calling function's body.

Single caller: the called function is inlined. Its body is substituted directly into the caller's instruction sequence. Multiple callers: the function is emitted as a subroutine. This prevents independent copies of its internal register state at each call site — a determinism guarantee, not an optimisation.

3.9 # in Rumus

```
(author writes)
? applyScale{value};
  result: value * #Config.scaleFactor
  ~result

(compiler expands to)
Config.scaleFactor ~ passthrough ~ applyScale.scaleFactor
? applyScale{value, scaleFactor};
  result: value * scaleFactor
  ~result
```

inside a Rumus function body means the value comes from a live Bija address. The compiler expands this to a Viesti inbound signal before emission. Rumus never reads Bija directly at runtime.

3.10 No Loops

Rumus contains no loop syntax. Every function is provably finite. Iteration is expressed as an explicit cycle in the Viesti signal graph, with Bija entities holding intermediate state between steps.

3.11 Memory Contract

Rumus functions operate exclusively in registers. No heap allocation occurs inside a Rumus function. When a function returns, every intermediate value is gone. The only way a result persists is if Viesti writes an output address to a Bija entity. Garbage collection is an inapplicable concept — there is nothing to collect.

3.12 Full Rumus Example

```
@Rumus
@rumus-riscv

(--- Roll vs defense: did the hit connect? ---)
? hitCheck{roll, defense};
  threshold: defense - 5
  hit: roll >= threshold ? 1 : 0
  ~hit

(--- Crit probability: luck-scaled ---)
? critCheck{luck, roll};
  chance: 60 + luck
  crits: roll * 6 > chance ? 1 : 0
  ~crits

(--- Strike: apply crit multiplier and armor reduction ---)
? strikeCalc{baseDmg, armor, isCrit};
  reduced: baseDmg - armor
  multiplied: isCrit = 1 ? reduced * 2 : reduced
  dmg: multiplied < 1 ? 1 : multiplied
  ~dmg

(--- Apply damage to target health ---)
? takeDamage{health, dmg};
  result: health - dmg
  health > 0 -> result >= 0
  remaining: result < 0 ? 0 : result
  ~remaining

(--- State machine: combat phase transition ---)
? transitionCombat{current, incoming};
  isIdle:   current = idle
  isCombat: current = combat
  isDead:   current = dead
  isDead !& isCombat
  isIdle -> incoming = combat
  isDead -> incoming = dead
  next: isDead ? current : incoming
  ~next
```

4. Viesti — The Signal Layer

4.1 Purpose

Viesti is the wiring of the application. It defines which data flows where. It does not compute. It does not store. It does not schedule. It routes.

Fudgel assumes a one-shot execution model. There is no built-in run loop, no frame tick, no event queue. A Fudgel program is a one-shot execution: data exists in Bija, signals fire, functions run, results are written back to Bija. Recurring execution is a Lifga library concern. The core language has no concept of time.

A signal resolves when its source address has data available. This is a pull model, not a push model. A function cannot execute until all of its parameter addresses have been populated by Viesti. If only some parameters are available the signal does not fire — no partial execution, no error. The condition is simply not yet met.

Viesti signals resolve in the order they are written in the file. Signal order is execution order. Writing signals is simultaneously writing the execution schedule. Deterministic execution order is a direct consequence of file structure, not something the runtime computes.

4.2 Sequential Signals — ~

```
source ~ function ~ target
```

The basic signal form. Three addresses, two tildes. Resolves completely before the next signal begins.

Part	Description
source	Named address on a Bija entity, or a parameter address on a function entity
function	A Rumus function entity
target	Named address on a Bija entity, or a parameter address on a function entity

4.3 Parallel Signals — ~~

```
(sequential seed)
Hero.luck ~ hitCheck.roll ~ Hero.luck
(parallel pool — all indented signals dispatched simultaneously)
Hero.luck ~~  critCheck.luck
    Battle.round ~ critCheck.roll
    Hero.attack ~~ strikeCalc.baseDmg
        Config.armorBase ~ strikeCalc.armor
        critCheck ~ strikeCalc.isCrit
    strikeCalc ~ takeDamage.dmg

(implicit barrier — main thread resumes here only when all pools above
are done)
Boss.health ~ takeDamage.health
```

The double tilde ~~ opens a parallel thread pool at the current execution depth. Signals declared under a ~~ are dispatched simultaneously to available threads. The main execution thread waits for all pools at the current depth to fully resolve before proceeding to the next sequential ~ signal. The barrier is implicit — defined by indentation structure, not by any keyword. Nesting works recursively; the tree resolves leaves-first.

4.4 Inline Parallel Chains

```
(inline chain)
A ~~ B ~~ C

(equivalent indented form)
A ~~
    B
    C

(two parallel pools separated by a sequential step)
A ~~ B ~~ C ~ D ~ E ~~ F
```

A ~ anywhere in an inline chain is a barrier — everything before it must resolve before it fires. In the example above: A, B, C are parallel; D runs sequentially after the barrier; E runs sequentially; ~~ opens a new pool with F.

> *Race condition warning: in A ~~ B ~~ C, B is simultaneously being written to and read from with no ordering guarantee. ~~ chaining is only safe when no address appears on both sides of adjacent ~~ operators. The IDE will flag overlapping addresses in ~~ chains.*

4.5 Parallelism Verification — @Peril

The Peril library is a compilation pass that verifies the independence claims of every ~~ pool. For each pool, Peril runs the pool's signals in both possible orderings against the full expanded Bija state, then compares outputs. Matching outputs prove independence. Differing outputs mean a dependency exists that was not declared — compilation halts with the conflicting signal path.

This is not a runtime check. It is a compiler-level correctness proof. @Peril is optional — a developer who is certain of their independence claims can omit it. Including @Peril means every ~~ in the application is compiler-verified rather than author-asserted.

The verification also exposes a useful diagnostic: if a program produces different outputs on two runs with identical input, there is exactly one cause in Fudgel — a ~~ where an address is being written and read without ordering. There is no other mechanism in the language that can produce non-determinism.

4.6 Dormant Signals

```
Player.health ~ checkCritical ~
```

A signal with no output target is dormant. Absent from the execution graph entirely — a compile-time known absence, not a runtime invalid state. Visible in the IDE graph as an unwired edge. Wiring an output activates it immediately without any other change.

4.7 Iteration via Signal Cycles

```
WorkQueue.current ~ processItem ~ WorkQueue.result
WorkQueue.result  ~ checkDone   ~ WorkQueue.isDone
WorkQueue.isDone  ~ onFalse     ~ WorkQueue.current
WorkQueue.isDone  ~ onTrue      ~ Output.final
```

Iteration is expressed as an explicit cycle in the Viesti graph. A signal targets a Bija entity that is itself the source of another signal, creating a feedback path. The cycle is fully visible in the graph. The runtime sees it. The author declared it. There is no hidden looping.

4.8 Conditional Routing

Viesti has no conditional syntax. A signal resolves when its source has data — unconditionally. Conditional branching is the result of a Rumus function writing to distinct Bija addresses. Separate signals are wired from each. Viesti routes from whichever address receives data. The decision was made inside Rumus.

4.9 Full Viesti Example

```
@Viesti

(--- Sequential: seed the hit check ---)
Hero.luck ~ hitCheck.roll ~ Hero.luck

(--- Parallel cluster: crit check + armor lookup ---)
Hero.luck ~~ critCheck.luck
    Battle.round ~ critCheck.roll
    Hero.attack ~~ strikeCalc.baseDmg
        Config.armorBase ~ strikeCalc.armor
        critCheck ~ strikeCalc.isCrit
    Boss.health ~~ takeDamage.health
        strikeCalc ~ takeDamage.dmg

(--- Sequential: apply damage ---)
takeDamage ~ Boss.health

(--- Sequential: phase transition ---)
Boss.health ~ phaseCheck.health
Config.maxHealth ~ phaseCheck.maxHp
phaseCheck ~ Boss.phase

(--- Dormant: xp formula not yet wired ---)
Battle.lastDmg ~ xpGain.dmg
Battle.round ~ xpGain.round
xpGain ~
```

5. Layer Agnosticism

The most important architectural principle in Fudgel is not the Trinity itself — it is that each layer is completely blind to the existence of the others.

Layer	Knows only how to...
Bija	Parse entities, properties, and boolean sub-entities. Store them. Display them as a tree.
Rumus	Accept named inputs, evaluate a sequence of expressions, write results to ~ output addresses, and return. It does not know where inputs came from or where outputs are going.
Viesti	Read a source address, invoke a function, write to a target address. It does not know what the function does, what types mean, or when anything should happen.
Libraries	Operate on the Bija state passed to them. They do not know other libraries exist.

This is not a design preference. It is what makes the system provably simple. If Viesti knew about Rumus semantics, you would need to version them together. If Bija knew about types, you would need to update the parser when type rules changed. Agnosticism means every layer can be understood, tested, and replaced in complete isolation.

5.1 The Library Compilation Pass

Because Bija stores all metadata booleans as plain sub-entities, libraries can participate in compilation by reading that state and acting on it before the PAF is emitted. This is how type checking, validation, and constraint enforcement work without any of those concepts existing in the core language.

The compilation pipeline is ordered:

- Bija parsing — produces a flat entity tree. No semantics assigned.
- Library passes — each participating library runs its Rumus functions over the entity tree in import declaration order. Any pass that returns false halts compilation and surfaces the failing entity path.
- PAF emission — the validated entity tree is compiled to binary format. Names are discarded. Addresses are assigned.

Libraries are architecturally independent — each pass is self-contained. That independence does not mean passes have no natural dependency order. That order is expressed in the @ import list — explicitly and visibly. A custom library that assumes another has already run must appear after it in the import list.

6. The Memory Model

The memory model follows directly from the separation of the Trinity.

Layer	Memory role and lifetime
Bija	The entire application heap. Entities are allocated at load time and persist until the application terminates or a signal writes empty to them. No dynamic allocation occurs at runtime.
Rumus	Registers only. Allocated at function entry, freed at function return. Nothing escapes unless Viesti explicitly writes the output to a Bija entity. Garbage collection is an inapplicable concept.
Viesti	The write path between Rumus and Bija. Not a memory region. A signal is a resolution operation, not a data store.

> *Invariant: a value exists in one of two states — stored in a named Bija entity, or in transit through a Rumus function. It cannot be in both simultaneously. It cannot be anywhere else.*

7. The Domain Libraries

The Fudgel standard library is composed of domain-specific libraries. Each library is agnostic: it does not know other libraries exist. It knows only how to operate on the Bija state it is given.

Libraries are domain-scoped semantic abstractions that compile directly to Bija entities, Rumus functions, and Viesti signals. There is no distinction at the machine level between library code and author code. Libraries receive no special compiler access and carry no runtime overhead beyond what their compiled output requires.

7.1 The @ Declaration

`@` has one universal meaning: bring this into scope. It applies equally to the Trinity and to domain libraries. The Trinity is not privileged at the syntax level — only at the architecture level.

A library file is a Fudgel source file. It compiles to the same Bija entities, Rumus functions, and Viesti signals as any other file. There is no intermediate compiled format. The source is the library. The compiler includes only what is actually referenced.

```
(standard library)
@Ehto

(custom or third-party)
@path/to/library
@../shared/myUtils
```

7.2 Libraries Are Inherently Open

Fudgel has no proprietary library model. This is not a licensing stance — it is a structural consequence. A library is a Fudgel source file. There is no intermediate compiled format, no opaque binary, no bytecode to distribute in place of source. The source is the library.

The conventional proprietary library model is architecturally impossible in Fudgel. An author who distributes a pre-compiled PAF is publishing an application, not a library. A library requires source because the import relationship requires it. Libraries accumulate openly by default.

7.3 Standard Libraries

Target Libraries

Pair with @Rumus to define what operations are available for a given execution substrate:

Library	Description
@rumus-riscv	Numeric arithmetic and comparison primitives for RISC-V execution. Current working target.
@rumus-wgsl	Planned. Shader operation primitives for GPU execution via WGSL.
@rumus-llvm	Planned. LLVM IR emission for broad hardware coverage.

Domain Libraries

Substrate-agnostic semantic abstractions built on top of whichever target is in scope:

Library	Meaning	Responsibility
Tipus	Type	User input handling
Ehto	Constraint	Type checking and validation
Primi	First	Memory management
Peril	Peril	Parallelism verification — proves ~~ independence at compile time
Lifga	Revive	Timing, sequences, animation — the only library that introduces time
Aquilla	Eagle	Visibility and framebuffer rendering
Kadro	Frame	Layout and structural templates
Smink	Makeup	Styling, formatting, and theming
Rezo	Network	High-performance network handling
Zapis	Record	File system and database I/O
Nësen	Nest	Dynamic entity allocation — operates in a separate memory pool

Library	Meaning	Responsibility
Giya	Guidance	Accessibility and semantic markers
Hloov	Change	Translation and localisation
Kani	Sound	Audio processing and playback
Pismo	Font	Text formatting and typography
Cead	Permission	User access control and security
Nyem	Squeeze	Compression and storage
Drysu	Tangled	Encryption and cryptographic safety

7.4 The Visual Library Stack

Kadro, Smink, and Aquilla are not graphics libraries in the traditional sense. They are semantic description layers that happen to produce visual output.

- **Kadro** answers *where is this and how big* — pure spatial arithmetic, entirely CPU-expressible.
- **Smink** answers *what does this look like* — property resolution over the entity tree, pure Rumus computation.
- **Aquilla** answers *what is visible and in what order* — a visibility and compositing pass over what Kadro and Smink already resolved.

By the time Aquilla finishes its pass the output is a resolved list of rectangular regions with known properties: position, size, colour, opacity, layering order. This is a framebuffer write problem, not a 3D render pipeline problem.

The rendering model is invalidation-based, not loop-based. A signal fires, Aquilla resolves the affected region, pixels update, execution stops. A Fudgel application with no state changes consumes essentially zero CPU — not polling at 60fps waiting for nothing to happen, genuinely idle until a signal fires. The signal graph already knows exactly what changed because the dependency graph was declared at compile time. There is no virtual DOM reconciliation, no full frame diff. The diff is free because it is the signal table.

This model scales to realtime applications and games. For workloads driven by discrete state changes — UI, strategy games, RPGs, simulations, turn-based games, and a large class of action games — the CPU framebuffer model is not a compromise. It is the correct architecture. The GPU pipeline wins for genuinely GPU-class workloads: thousands of entities with per-vertex transformations, particle systems, volumetric rendering.

7.5 Physics and Simulation

The same signal-driven model that makes Aquilla's rendering accurate makes physics simulation accurate in a way traditional engines are not. Conventional physics engines approximate because they must — the render loop allocates a time slice to physics, so the simulation runs fewer iterations than physical accuracy would require and accumulates error.

Fudgel has no frame budget. A physics signal fires, the simulation runs Rumus functions to completion — however many iterations convergence requires — and writes the result to Bija. The CPU is not racing a clock. The physics can be as accurate as the algorithm, not as accurate as the budget allows.

Precalculation and baking are first-class citizens of the model. Because Bija is fully declared state, the boundary between precalculated and live is explicit and visible in the source. A developer can run the full accurate simulation at authoring time, write the results into Bija entity states, and at runtime apply only the delta between the baked path and actual player interaction. For rigid body simulations, ballistics, fluid in contained spaces, and any physics with bounded interaction surfaces, this model produces genuinely accurate results from a modest CPU.

8. The PAF — Published Application Format

The PAF is the compiled, binary-efficient output of a finalised Fudgel application. It is a shipping format, not a development format. Development happens in the Fudgel IDE against a live entity and signal graph.

8.1 Memory Model

PAF entity blocks are variable width. There is no way to know in advance what data a Bija entity will contain — fixed width blocks would either waste space or impose an artificial ceiling on entity complexity. Variable width requires an index. The PAF header maps entity addresses to byte offsets. Lookup is O(1) via the index rather than O(n) traversal.

The PAF declares its total memory footprint upfront in the header. The runtime allocates this once on load and does nothing further for core PAF space. There is no allocator at runtime — the compiler made every allocation decision. The runtime honours what the compiler decided.

8.2 Header Structure

```
[total_memory_footprint]   — runtime allocates this once
[entity_count]             — length of the index
[entity_start_offsets]     — flat array, one value per entity
[signal_count]             — length of signal table
[signal_table]             — flat array of address pairs
[data_region_offset]       — where bulk bytestreams begin
```

Three regions follow the header: the index, the block region containing variable-width entity blocks navigated via the index, and the data region containing raw bytestreams for large values referenced by offset from within blocks.

8.3 PAF Space and Nësen Space

PAF space is entirely static, entirely compiler-decided, entirely predictable. A program that does not import Nësen has provably no dynamic allocation, no GC, no runtime memory management of any kind. That is an auditable guarantee, not a claim.

Nësen — the dynamic entity allocation library — operates in a separate memory pool outside PAF space with its own allocator. PAF addresses and Nësen addresses occupy distinct ranges. A signal destination is unambiguously one or the other from the address value alone — no type tag, no runtime check. A signal pointing into PAF space is always statically inspectable. A signal pointing into Nësen space is visibly dynamic.

8.4 Format Properties

A PAF file contains no source names, no type information beyond memory layout requirements, and no runtime interpreter. It is loaded by the Fudgel Runtime, which maps it into memory and begins executing from the first signal declared in the application's @Viesti section. Execution proceeds in signal declaration order from there.

Distributing a PAF is publishing an application. It is not a library distribution mechanism. The PAF format specification is a separate document and is currently under development.

9. The Fudgel IDE

9.1 Architecture

The Fudgel IDE is a multitab environment. The primary tab presents the application as a live schema — a node graph of the Bija entity tree and Viesti signal graph that actually runs. It is not a mockup. It is the application, rendered as a readable structure, executing in real time or at controllable speed.

The graph view is strictly read-only. The canonical source of truth is always the code the developer writes. This is a deliberate constraint: visual editors that are also the source of truth tend to produce proprietary formats, locked-in tooling, and code hostile to standard workflows. By keeping the graph read-only, Fudgel retains full compatibility with version control, code review, and the entire text-based tooling ecosystem.

This constraint produces real friction. A developer who wants to restructure their signal graph by dragging nodes cannot. Reorganising a complex application means editing source. That is the point. The friction of editing source is the friction of understanding what your program actually does.

This is only possible because Fudgel's source structure and execution structure are the same thing. There is no translation to visualise. The graph renders what is already true about the program.

9.2 Library Tabs

Each domain library gets its own IDE tab. The render library tab shows the visual layout. The signal library tab shows timing and event flow. The type library tab shows constraint validation state across the entity tree. This segments complexity by domain — nobody has to look at the full graph at once unless the task requires it.

9.3 Code-to-Graph Navigation

Selecting a node in the graph view jumps to the Bija entity or Viesti signal that defines it. Editing the source updates the graph immediately. This linkage is what makes the IDE a genuine development environment rather than a static viewer.

9.4 MVP Visualiser

The full IDE is gated on library maturity. The immediate priority is an MVP visualiser that parses Fudgel source and renders the signal graph, showing data movement in real time or at controlled speed. This validates the core Trinity implementation and makes the language comprehensible to people who have not read the spec.

10. Build Order

Stage	Deliverable and rationale
1. Core compiler	Parser for Bija, Rumus emitter targeting RISC-V, Viesti graph builder including parallel pool resolution. The minimum needed to produce a valid PAF from source.
2. MVP visualiser	Read-only graph renderer showing the live Bija entity tree, Viesti signal flow, and parallel execution clusters. Makes the language legible to contributors.
3. Base libraries	Ehto (type/constraint), Primi (memory), Lifga (timing), and a base iteration library. Enables real programs to be written.
4. Domain libraries	Aquilla, Kadro, Smink, and the remaining domain-specific libraries, developed in order of need.
5. Full IDE	Multitab environment with domain library tabs, built reactively from what each library proved it needs to show.

11. Architectural Invariants

The following invariants must hold in any valid Fudgel application. They are not conventions. A Fudgel compiler must reject any application that violates them.

- Every Fudgel file must declare its Trinity imports explicitly. @Bija, @Rumus, and @Viesti are required declarations. A file's @ block is its interface declaration.
- @ has one universal meaning: bring this into scope. The Trinity DSLs use the same syntax as domain libraries.
- A file's type is defined by its imports. An application file declares all three Trinity layers. A library file declares @Bija and @Rumus but not @Viesti. A data file declares only @Bija.
- The first signal declared under @Viesti is the application entry point. Execution begins there and proceeds in signal declaration order.
- Bija contains no unresolved references. Every address is fully determined at load time.
- Bija values are always atomic. A value is a literal, a reference, a shorthand token, or empty. Never an expression.
- The * list marker creates named sub-entities with empty values. Names are the data. The - list marker creates numbered sub-entities with their values. Neither construct exists in the compiled output.
- The ! multiplier resolves at parse time to N independent named instances. The count must be a literal integer.
- A Fudgel program is deterministic by construction. Non-determinism can only be introduced explicitly through a library.
- Bija has no privileged keywords. All metadata booleans are plain sub-entities. Their meaning belongs to the library that reads them.
- Rumus function parameters are declared using {} boolean shorthand. They expand to sub-entities with value true. Viesti overwrites them before execution. A function cannot run unless all parameters have been supplied.
- The only externally addressable values on a Rumus function entity are its parameter addresses and its ~ declared output values. All body properties are ephemeral and do not exist in the PAF.
- Body property names in Rumus are discarded at emit time and become positional register slots. Body property count determines register demand. Overflow is undefined behaviour.
- A Rumus function called by a single caller is inlined at emit time. A function called by multiple callers is emitted as a subroutine. This is a determinism guarantee, not an optimisation.
- Rumus never reads Bija directly at runtime. # in Rumus is always expanded to a Viesti signal at compile time.
- Viesti signals resolve in the order they are written in the file. Signal order is execution order.
- A signal resolves when its source address has data available. This is a pull model. A function does not fire until all parameters are populated.
- Sequential signals (~) resolve completely before the next signal begins.
- Parallel signals (~~) are dispatched simultaneously to available threads. The main thread waits for all pools at the current depth before the next sequential signal. The barrier is implicit, defined by indentation.

- ~~ chaining is only safe when no address appears on both sides of adjacent ~~ operators. Overlapping addresses produce a race condition.
- A signal with no output target is dormant. It is absent from the execution graph entirely — a compile-time known absence, not a runtime invalid state.
- Timing, scheduling, animation, and recurring execution are exclusively Lifga library concerns. Viesti has no concept of time.
- Every Rumus function is finite. No function may directly recurse or contain cycles in its call graph.
- Iteration exists only as explicit cycles in the Viesti signal graph.
- The only mechanism that writes to Bija at runtime is a Viesti signal carrying a function output value.
- No layer knows another layer exists. Bija does not know Rumus exists. Rumus does not know Viesti exists. Libraries do not know other libraries exist.
- Library compilation passes run in import declaration order. Import order is pass order.
- There is no compiler optimisation pass. Performance is a product of design quality.
- Rumus core is substrate-independent. Target libraries (@rumus-riscv, @rumus-wgsl, etc.) declare substrate-specific instruction mappings. The current working target is @rumus-riscv.
- The Rumus standard operation set covers five domains: arithmetic (+ - * / % and unary -), comparison (= != < > <= >=), boolean (& | !), conditional (a ? b : c), and formal logic (-> <-> ^ !& !| <-). Bitwise operations are hardware-specific and belong in a systems target library.
- Formal logic operators (->, <->, ^, !&, !|, <-) are assertions, not expressions. They state relationships that must hold at function evaluation. If an assertion fails, execution halts with the function path and the specific assertion that failed.
- The conditional a ? b : c produces a value through pure transformation of inputs. It is valid only when both branches produce values of the same kind. Any selection that routes to meaningfully different behaviours belongs in Viesti.
- File I/O, database access, and session persistence are exclusively Zapis library concerns.

Appendix — Quick-Reference Card

Bija Symbols

Token	Meaning
EntityName;	Entity declaration
name: value	Property
name: empty	Explicit empty value
#Entity.prop	Alias (same address, resolved at compile time)
=Entity.prop	Value copy (copied at parse time, source forgotten)
| v1, v2 |	Positional override
!N	Multiplier — stamp N instances

Token	Meaning
-	Ordered list marker
*	Unordered list marker
{flag}	Metadata boolean
[...]	Literal string (no parsing inside, may span lines)
(...)	Comment

Rumus Symbols

Token	Meaning
? Name{p1, p2};	Function declaration
name: expr	Named binding (let)
~name	Output declaration
#Entity.prop	Live Bija read (compiler expands to inbound signal)
+ - * / %	Arithmetic
-x	Unary negation
= != < > <= >=	Comparison (= is equality, not assignment)
& | !	Boolean AND / OR / NOT
a ? b : c	Conditional
a -> b	Implication assertion
a <-> b	Biconditional assertion
a ^ b	Exclusive or assertion
a !& b	NAND assertion
a !| b	NOR assertion
a <- b	Converse implication assertion

Viesti Symbols

Token	Meaning
A ~ B ~ C	Sequential signal chain
A ~~ B	Parallel signal (concurrent with peer ~~ signals)
A ~ B ~	Dormant signal (no target)